National University of computer and emerging sciences

Lab 07: Dup System Calls

The dup and dup2 system calls in Linux are used to duplicate file descriptors.

dup System Call:

The dup system call duplicates an existing file descriptor.

The new file descriptor returned by dup is the lowest-numbered unused file descriptor.

Both file descriptors (the original and the new one) refer to the same open file description, so they share the same file offset and file status flags.

Syntax:

int dup(int oldfd);

Example:

```c
int main() {
    int fd = open("test.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    // Duplicate the file descriptor
    int fd_copy = dup(fd);
    if (fd_copy < 0) {
        perror("dup");
        return 1;
    }

    // Write using both descriptors
    write(fd, "Original FD\n", 12);
    write(fd_copy, "Duplicated FD\n", 14);

    // Close both descriptors
    close(fd);
    close(fd_copy);

    return 0;
```

The program opens a file test.txt with write access.

It duplicates the file descriptor using dup, creating a new file descriptor (fd_copy).

Both fd and fd_copy refer to the same file, so writing through either affects the file.

The file is written using both descriptors.

Both descriptors are closed.

Output:

```
$ cat test.txt
Original FD
Duplicated FD
```

dup2 System Call:

dup2 allows you to specify the new file descriptor. It duplicates the first file descriptor into the second file descriptor.

Syntax:

int dup2(int oldfd, int newfd);

Example:

```c
int main() {
    int fd = open("test.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    // Duplicate fd to stdout (1)
    if (dup2(fd, 1) < 0) {
        perror("dup2");
        return 1;
    }

    // Write to stdout (which now points to the file)
    printf("This will be written to test.txt instead of stdout.\n");

    close(fd);

    return 0;
}
```

The program opens a file test.txt for writing.

It uses dup2 to duplicate fd onto stdout (file descriptor 1). 0 is for stdin (Standard input i.e keyboard), 1 for stdout (Standard output), 2 for stderr (Standard error).

Any write to stdout (like printf) will now go to test.txt instead of the terminal.

The output is redirected to the file.

The dup2(fd, 1) command duplicates the file descriptor fd onto file descriptor 1, which is normally stdout. This means any output that would normally go to the terminal (stdout) will be redirected to the file represented by fd.

Output:

```
$ cat test.txt
This will be written to test.txt instead of stdout.
```

In dup2(fd, X), the second argument (X) can be any valid file descriptor. Here are the common values you might use:

Common File Descriptors:

0 – Standard Input (stdin): Normally, this reads input from the terminal. If you redirect this to a file using dup2(fd, 0), the file will become the input source.

1 – Standard Output (stdout): Normally, this outputs to the terminal. Using dup2(fd, 1), you can redirect the output to a file.

2 – Standard Error (stderr): Normally, error messages go to the terminal. You can redirect errors to a file using dup2(fd, 2).

Example using stdout and stderr:

```c
int main() {
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    // Redirect stdout to file
    dup2(fd, 1);

    // Redirect stderr to file
    dup2(fd, 2);

    printf("This is standard output (stdout)\n");
    fprintf(stderr, "This is an error message (stderr)\n");

    close(fd);
    return 0;
}
```

dup2(fd, 1): Redirects standard output to the file output.txt.

dup2(fd, 2): Redirects standard error to the same file.

Output in output.txt file:

```
This is standard output (stdout)
This is an error message (stderr)
```

Both the standard output and the error message are written to `output.txt` instead of the terminal.

Example: Redirecting stdin from a File:

This example opens a file for reading and then uses dup2 to redirect stdin so that when you use functions like scanf or fgets, the input comes from the file instead of the keyboard.

```c
int main() {
    // Open a file for reading
    int fd = open("input.txt", O_RDONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    // Redirect stdin (0) to the file
    dup2(fd, 0);

    // Read from stdin (which is now the file) using fgets
    char buffer[100];
    fgets(buffer, sizeof(buffer), stdin);  // This reads from "input.txt"

    // Print the line read from the file
    printf("Read from file: %s", buffer);

    // Close the file
    close(fd);

    return 0;
```

The program opens a file called input.txt in read-only mode.

dup2(fd, 0) redirects the standard input (file descriptor 0) to the file.

fgets(buffer, sizeof(buffer), stdin) reads from the file instead of the terminal.

The program prints the content it read from the file.

Contents of input.txt:

```
Hello, world!
```

Output:

```
Read from file: Hello, world!
```

When the program reads from stdin, it actually reads from input.txt because of the redirection with dup2(fd, 0).

The file's content (Hello, world!) is read and then printed to stdout.

Tasks:

1. Use dup() to duplicate a file descriptor for a file, store it in a variable fd_dup, then use fd_dup in dup2() to copy it into stdout and redirect the output of printf to the file. It means if we write printf("Helllo world"); then it should not display on terminal but instead it should be written into file.

2. Use `dup2()` to duplicate the file descriptor for `stdin`, so input can be taken from a file instead of the terminal.
   Use fgets(buffer, sizeof(buffer), stdin);
   Usually when we use fgets(buffer, sizeof(buffer), stdin); then it takes input from stdin which is set to keyboard so it takes input from keyboard and stores it in buffer variable but when you duplicate the file descriptor of a file into stdin then it will take reads data from a file and stores it in buffer.

3. Redirect stderr to a file using dup() and simulate an error message.

4. Use unnamed pipe to send data from child to parent and parents then prints it. Don't use write() system call instead use printf so that if you write printf("Messagge from child"); then it shouldn't be displayed on console but is should be written into write end of unnamed pipe and parents then reads it using read end of pipe and displays it on console.

Task Submission Guidelines:

1. **Make a word document and paste the code with output's screenshot there and save it as pdf or odt.**
2. **Include your name and roll no. at the front page.**
3. **Files other than pdf or odt will not be accepted and will be marked as 0.**
4. **Submit you C/C++ files as well without zipping the folder.**